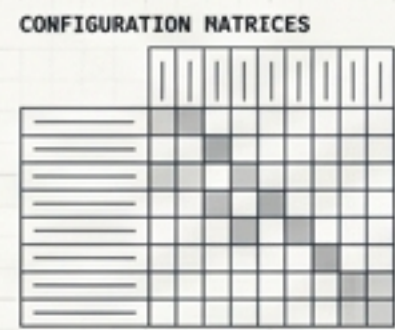


Orchestrating the Vision 2026 Cloud Baseline

A Deep-Dive into Environment Configuration, Variable Orchestration, and Terraform Module Dependencies

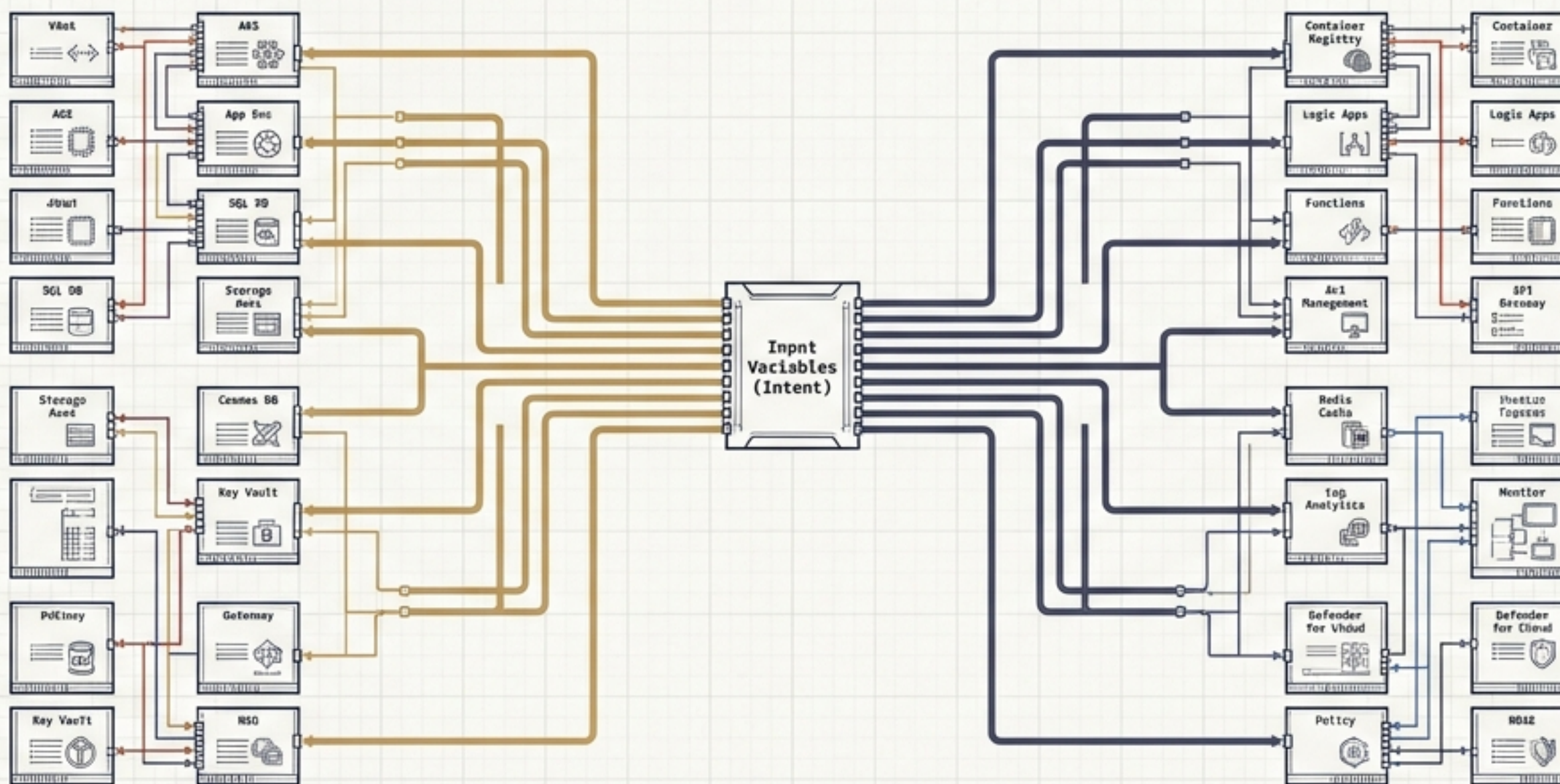


Key	Value
env	"prod"
region	"br-us1-1"
cost_center	"1232"
cost_cau	"prod"
sess	"prod"
session	"prod"

Key	Value
vnet_cidr	us-0045
db_version	00050-1
db_2kn	NS-8841
akt_versions	23
db_2kax	00-0001

```

graph LR
    Vlet[Vlet] --> AKS[AKS]
    SNL[SNL DB] --> AppSec[App Sec]
    KV[Key Vault] --> All[All]
  
```

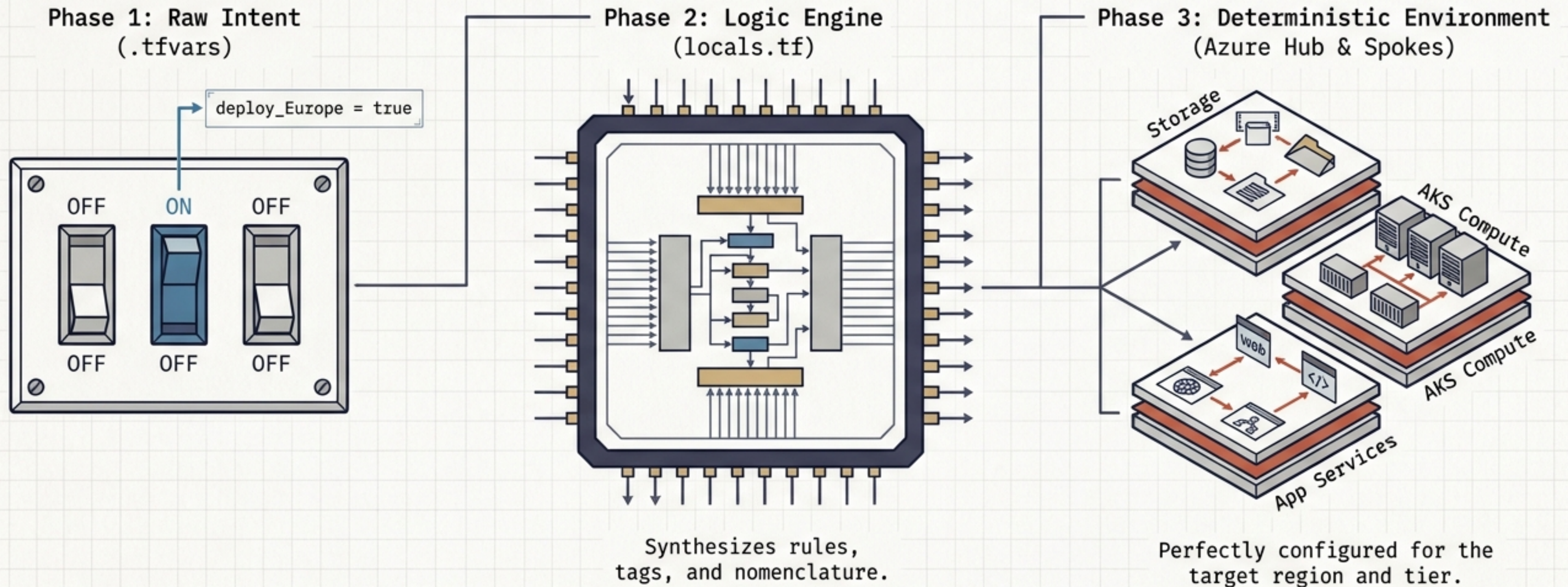


```
graph TD; A[VALIDATE INTENT] --> B[PLAN GENERATION]; B --> C[APPLY REACUREES]; C --> D[VERIFY STATE]; D --> A; B --> C; E[AUDIT LOGS] --> C;
```

Paik
modules/netuncking/vnet
modules/computo/aka
modules/ecomnce/aks
modules/data/cgl
modules/data/agl
modules/securitay/heyvault

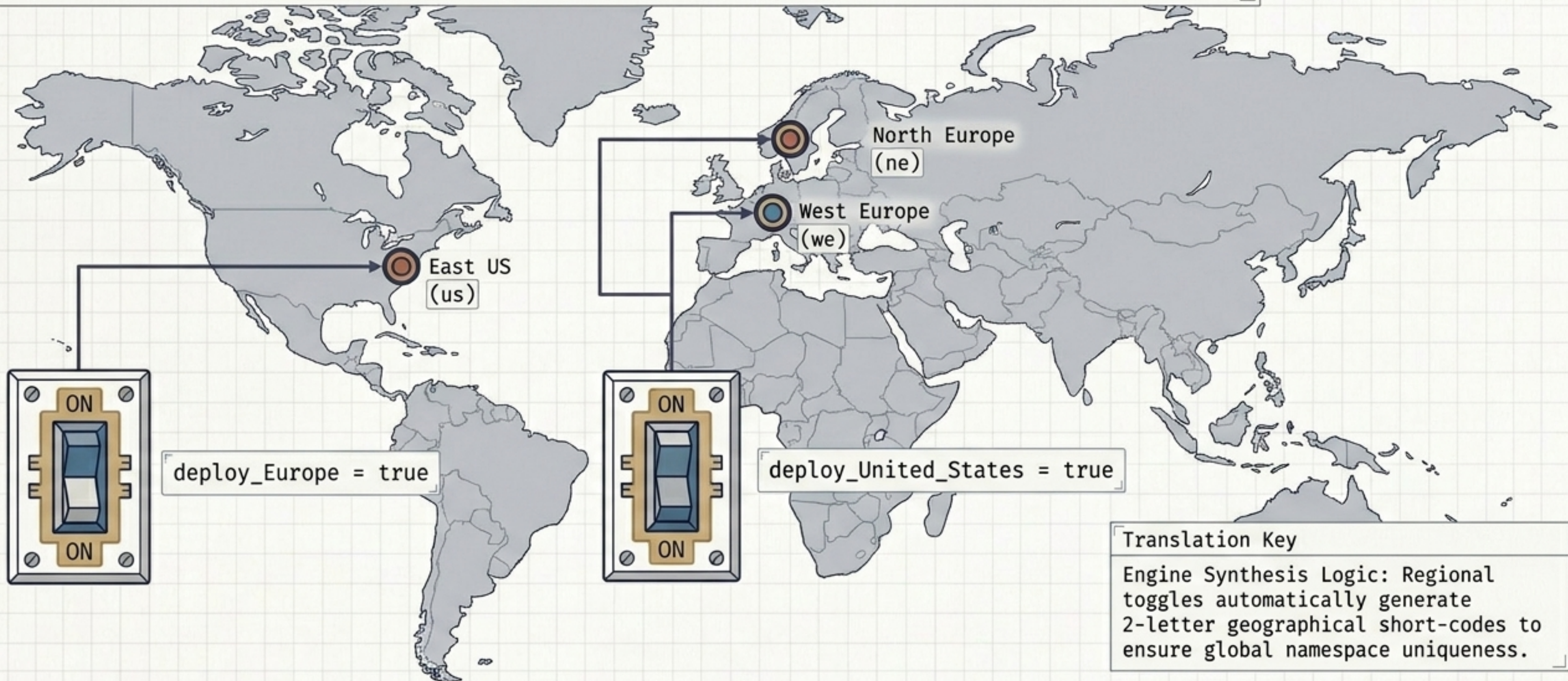
The DRY-First Philosophy: Infrastructure as Pure Software

A Multi-Layer Logic Engine translates high-level architectural intent into hundreds of low-level Azure resources. Zero code duplication. Variables act as strategic Feature Flags.



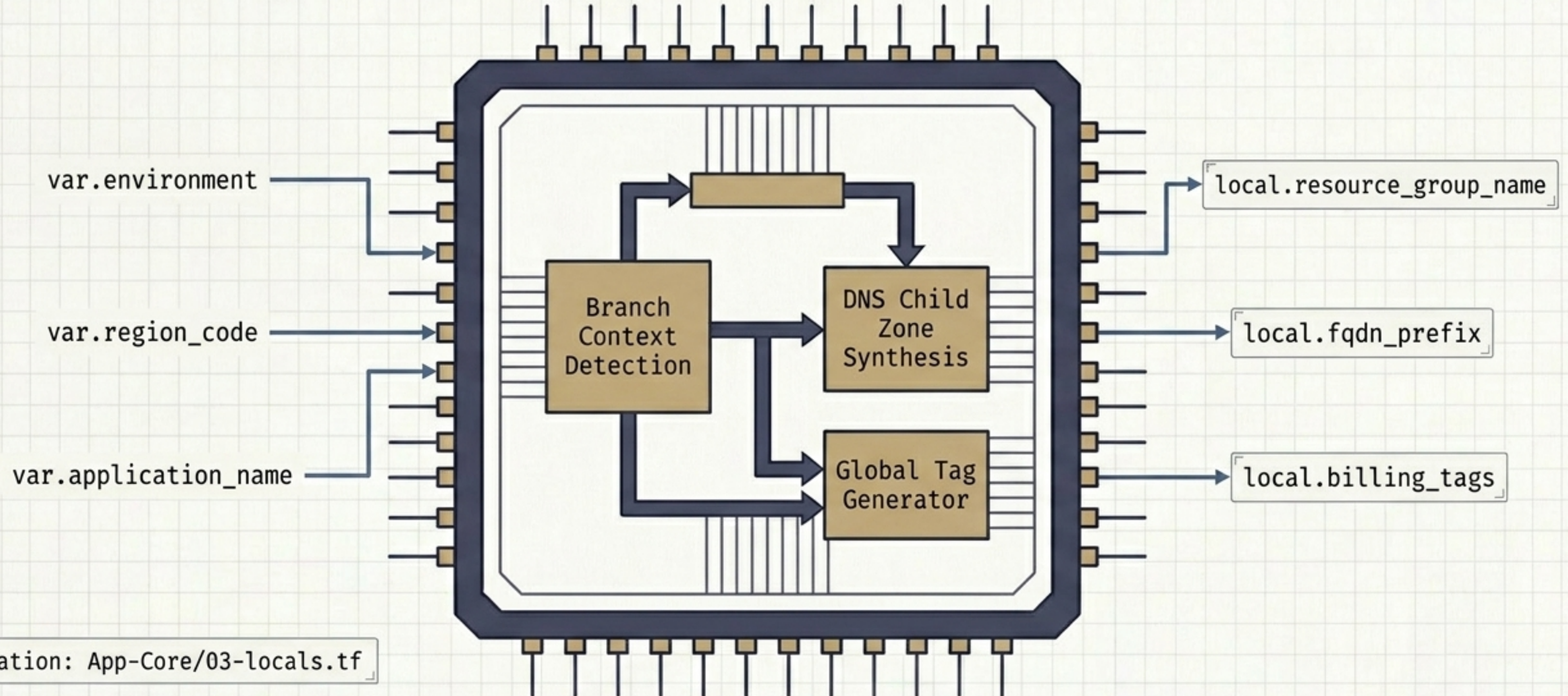
The Input Layer: .tfvars as Architectural Feature Flags

Multi-Region Fan-Out and Global Scale are achieved by passing single boolean flags that activate entire geographical stacks independently.

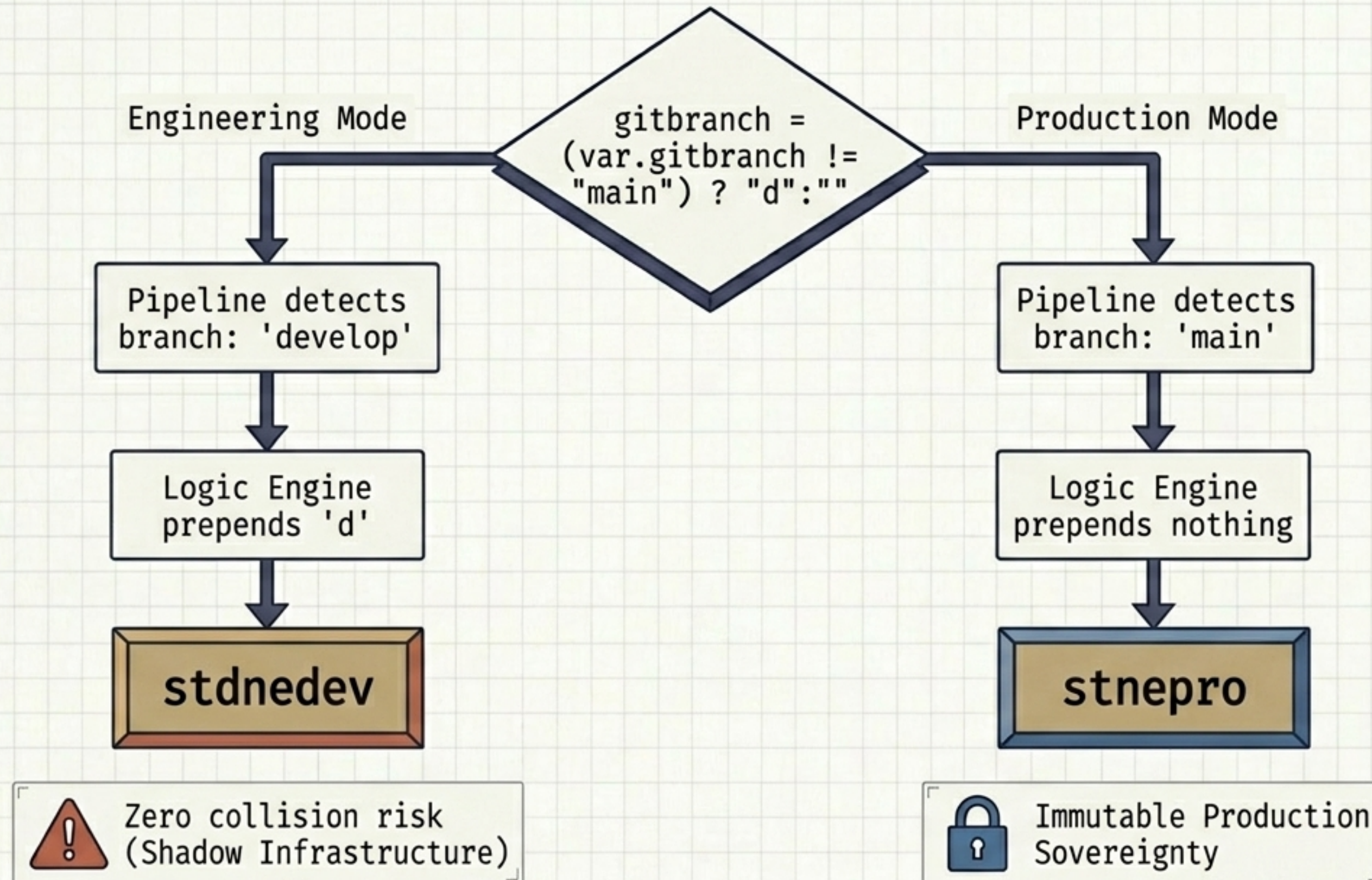


The Brain of the System: The locals.tf Logic Engine

Raw variables are never passed directly to Terraform resources. They act as calculation buffers to prevent namespace collisions.



Branch-Aware Variables: The "d" Prefix Logic



Environment Variable Routing: ENG vs. PRO Tiering

Calculating "sharedinfra_environment" to enforce absolute billing, quota, and security isolation via a Spoke-Per-Tier model.

Engineering (ENG)			Production (PRO)		
1	Variables Mapped:	DEV, QA, UAT, PRE	Variables Mapped:	PRO, DEMO, RES	
2	Subscription Role:	Enterprise DevTest	Subscription Role:	Enterprise Prod	
3	Service Connection Identity:	svccon-ent-dev	Service Connection Identity:	svccon-ent-pro	
4	SPN Permission Levels:	Contributor + User Access Administrator	SPN Permission Levels:	Contributor (Strict JIT required)	

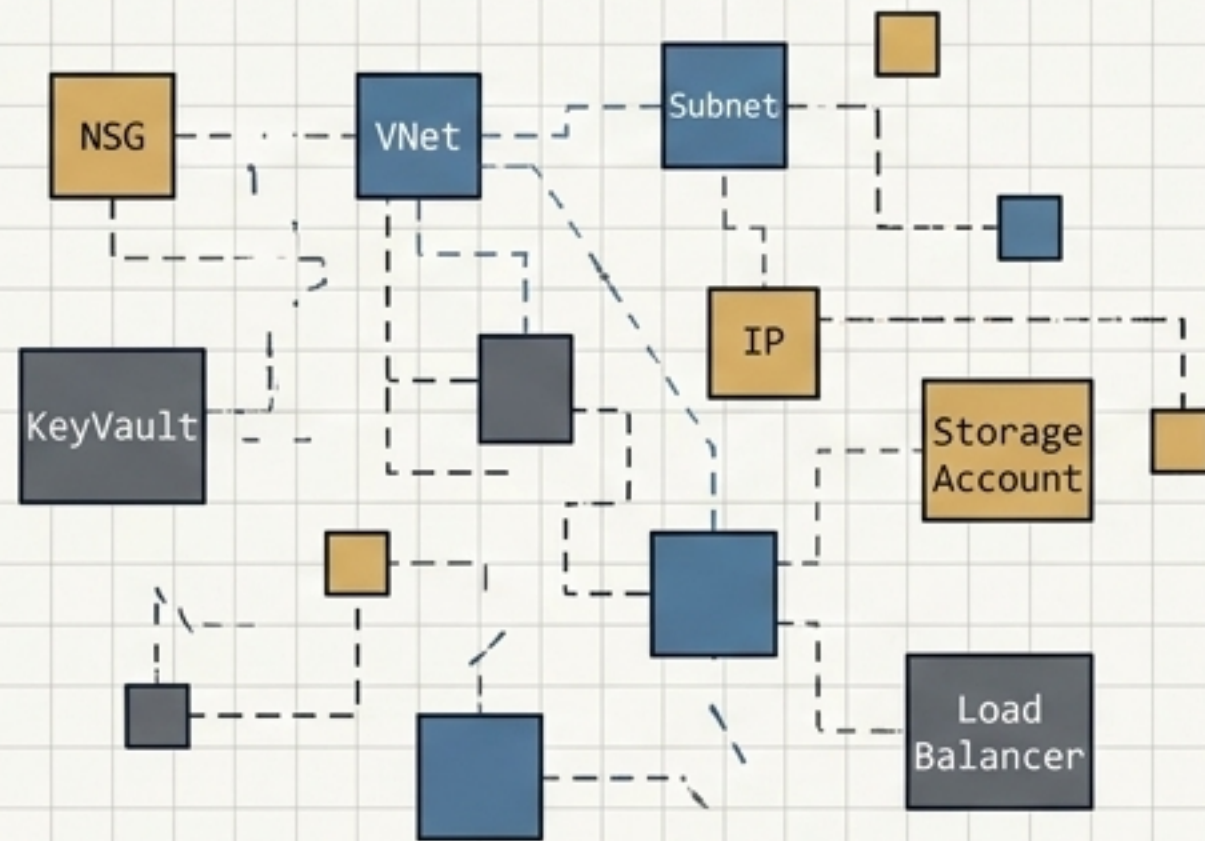


Strategic Routing Insight: Demo and Research environments are mapped to the PRO tier to enforce production-grade security standards (Firewall, WAF) on experimental public-facing assets.

Module Architecture: Composite vs. Atomic Design

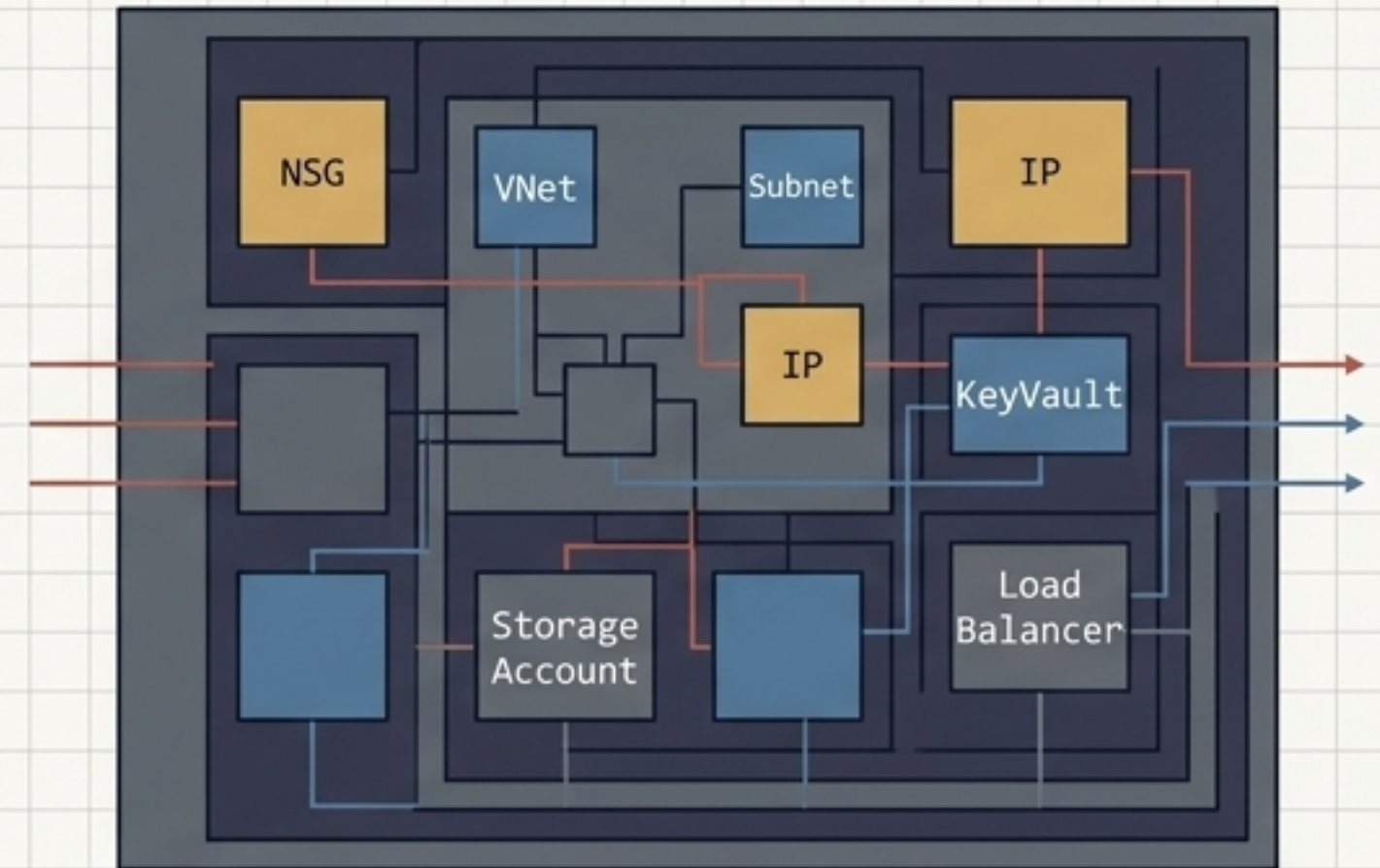
Variables are fed into massive, domain-driven Composite blocks to ensure strict standardization and accelerated enterprise delivery.

Atomic Architecture (Standard)



- 10 to 20 separate module calls per stack
- Loose standards reliant on caller execution
- Heavy, scattered maintenance overhead
- Suitable for fragmented giant enterprises

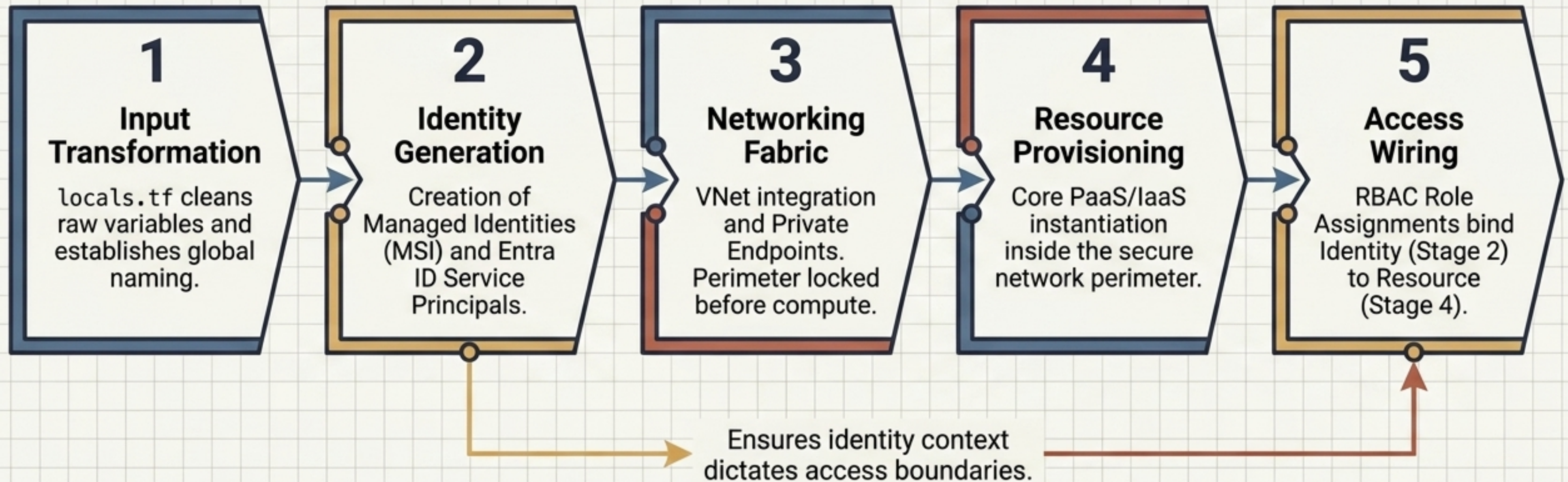
Composite Architecture (This Repository)



- 1 module call deploys a Full Architectural Tier
- Strict organizational standards baked internally
- Highly focused domain-driven maintenance
- Optimized for agile, mid/large multinational delivery

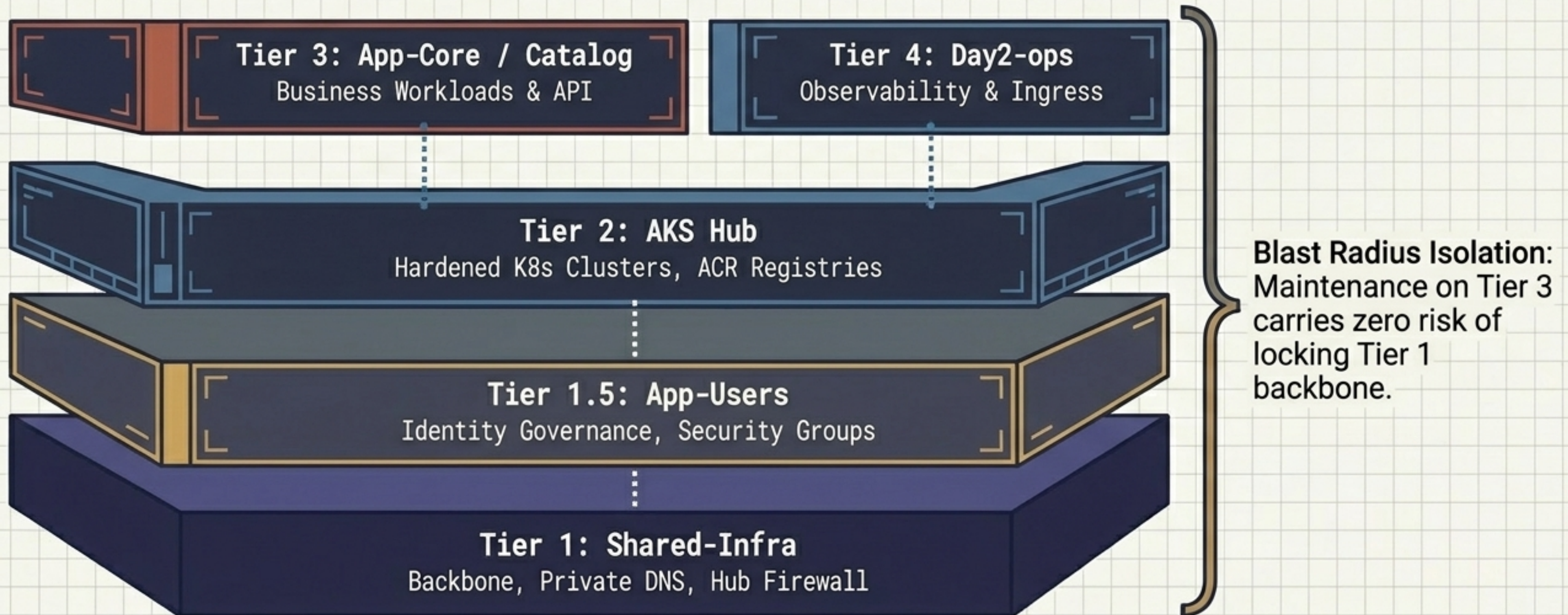
The 5-Stage Internal Module Lifecycle

Zero-trust security is physically wired into the resource at the exact moment of creation through a chronological internal variable flow.



The Macro Dependency Chain (Logical Hierarchy)

Mandatory deployment sequence isolating partial lifecycles to guarantee dependent fabrics exist before compute orchestration.



IaC Sovereignty: Referential Attribute Linking

Managing dependencies via Implicit Dependency Graphs prevents "Chicken and Egg" data source errors. Hardcoded strings are strictly eliminated.

Bad Practice: Spaghetti Data Blocks

Structural IDE window:

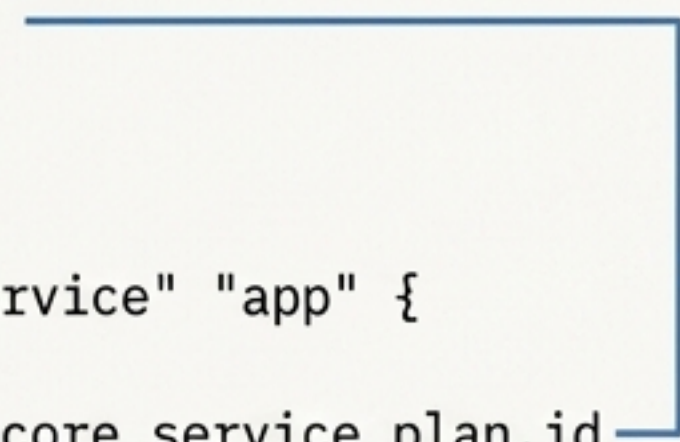
```
data "azurerm_subnet" "lookup" {  
  name = "snet-app-prod"  
}  
  
resource "azurerm_app_service" "app" {  
  subnet_id = data.azurerm_subnet.lookup.id  
}
```

Relies on rigid string lookups. Breaks if execution order falters.

Good Practice: Implicit Graph Generation

Structural IDE window:

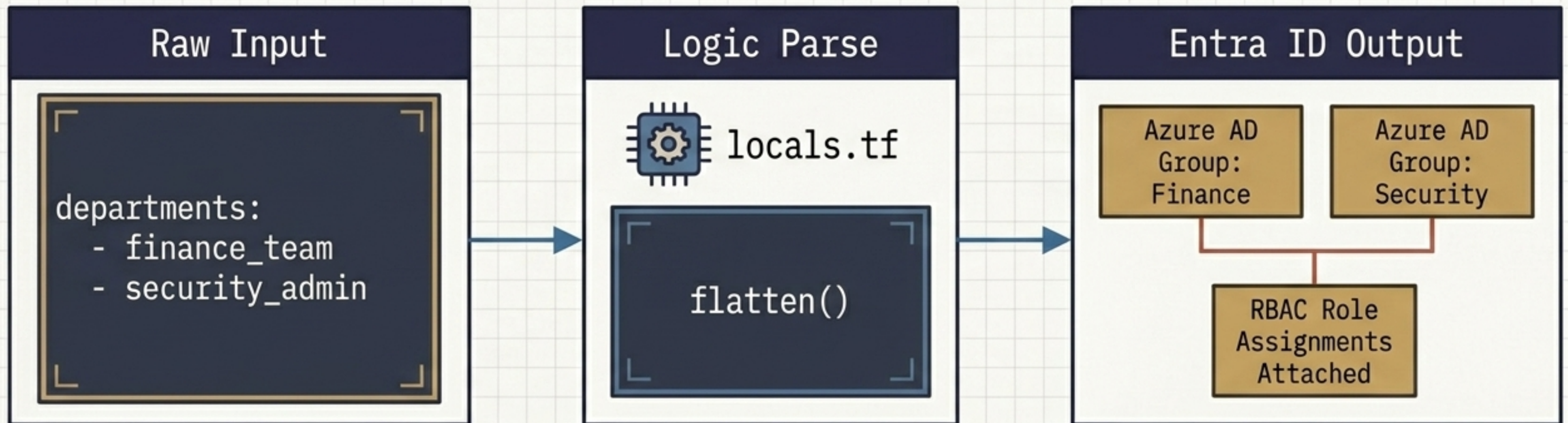
```
resource "azurerm_service_plan" "appcore_service_plan" {  
  ...  
}  
  
resource "azurerm_app_service" "app" {  
  service_plan_id =  
    azurerm_service_plan.appcore_service_plan.id  
}
```



Directly passing output attributes as inputs guarantees absolute state integrity.

Advanced Configuration: YAML-Driven IaC

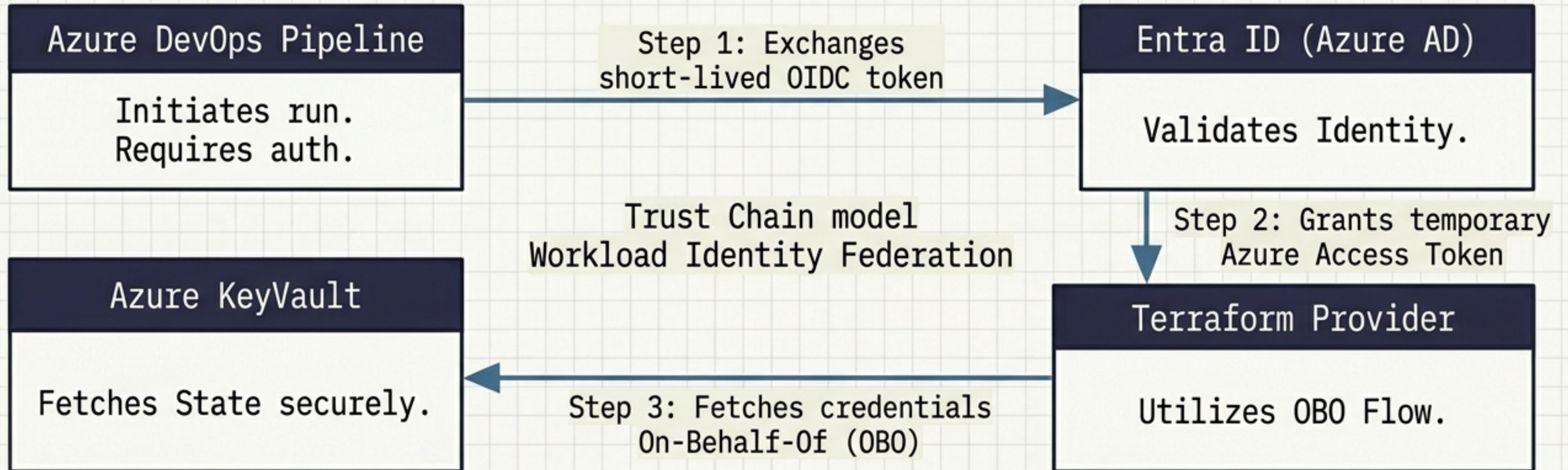
Moving beyond flat .tfvars files. Complex nested data structures enable non-Terraform engineers to govern identity provisionally.



Module Application: Tier 1.5 App-Users

Secretless Pipeline Configuration (Zero-Secret)

Pipeline variables integrate with identity. The toggle "use_oidc = true" eliminates the need for long-lived Service Principal Keys.

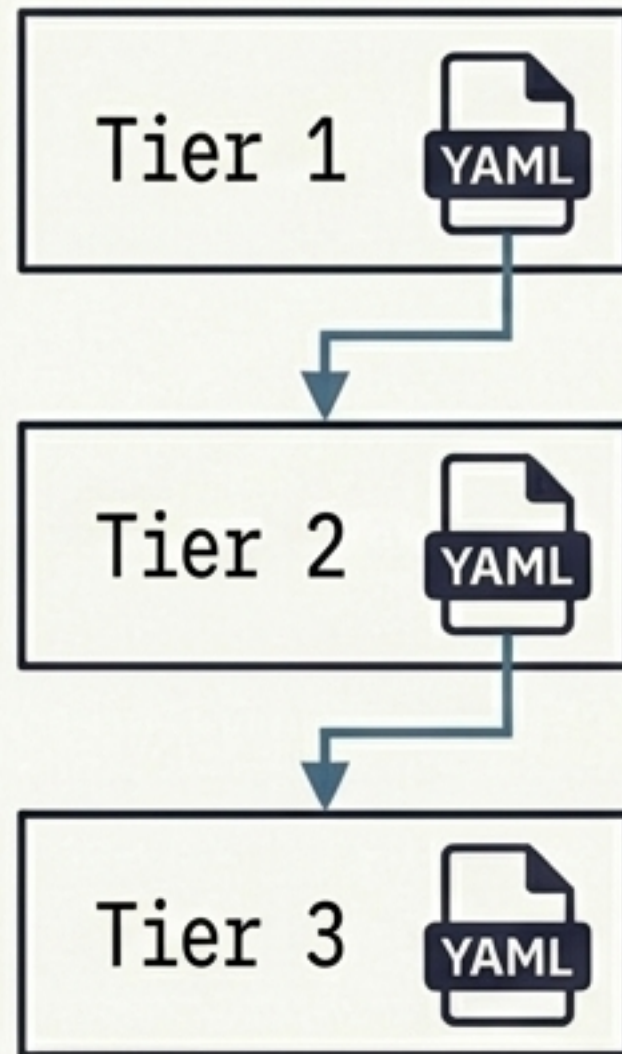


**Zero credentials stored in repository.
Zero credentials on build agents.**

Orchestration Strategy: Present vs. Future (2026)

The architectural evolution of executing variables and dependencies at scale.

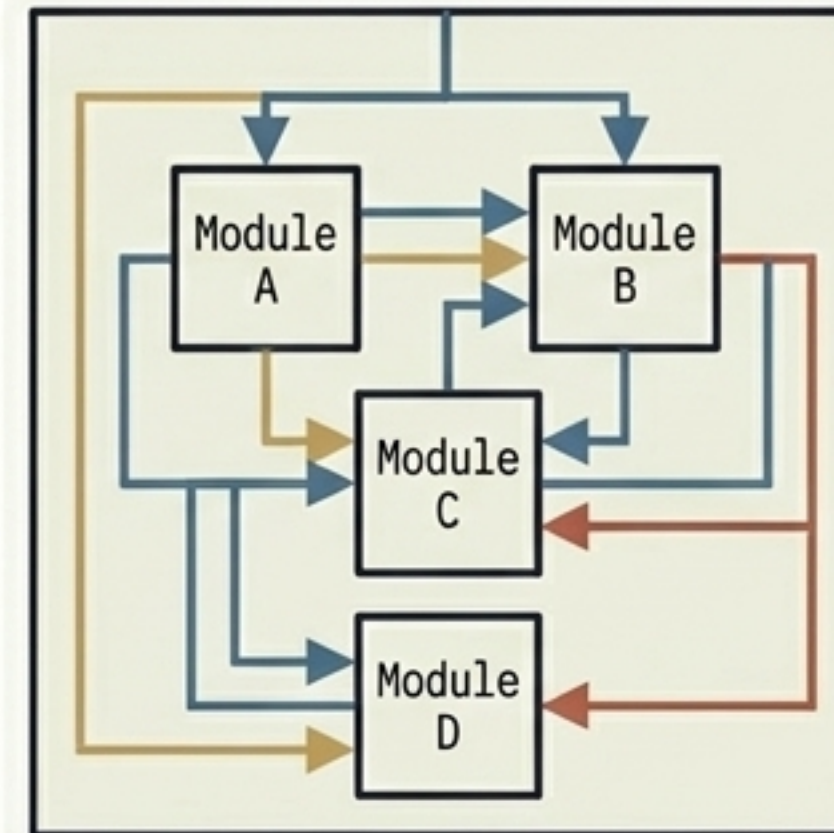
Current: Pipeline-Driven YAML



- External Azure DevOps orchestration
- Strict sequential execution
- Requires remote data lookups across isolated layers

Future (2026): Native HCL Stacks

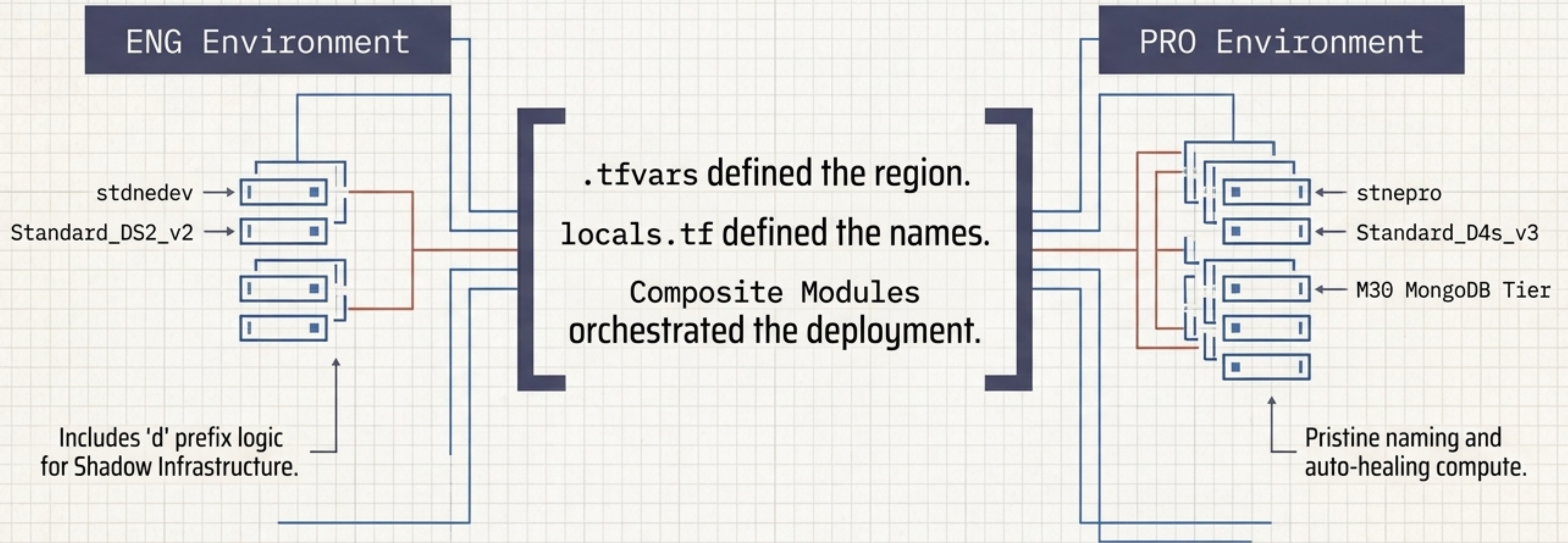
Native Terraform Stack



- Autonomous dependency management
- Automatic parallelization
- Single-logic unified planning

Synthesis: The Deterministic Environment Map

Absolute **parity** between Engineering and Production via identical codebase orchestration.



Engineering Onboarding & Ignition

Prerequisites required to feed the variable engine and execute initial deployment.

1. Core Subscriptions

- ☐ 2x Enterprise Subscriptions deployed
- ☐ Strict ENG and PRO isolation configured

2. Identity Privileges

- ☐ Global Admin rights verified
- ☐ Initial SPN / OIDC trust chain established

3. CLI Tooling Versions

- ☐ Git $\geq$ 2.40
- ☐ Terraform $\geq$ 1.5.2
- ☐ kubelogin & helm installed

4. Development Environment

- ☐ VSCode installed
- ☐ HashiCorp Terraform Extension activated via optimized settings.json



ACTION REQUIRED: Reference SRE / Runbook documentation to initiate Tier 1 (Shared-Infra) provisioning.